

```
!apt-get install -y ffmpeg
!pip install -q pydub librosa soundfile scipy matplotlib numpy
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ffmpeg is already the newest version (7:4.4.2-0ubuntu0.22.04.1).
0 upgraded, 0 newly installed, 0 to remove and 41 not upgraded.
```

```
import requests
from pydub import AudioSegment

# Using a new, publicly accessible MP3 URL
audio_url = "https://www.learningcontainer.com/wp-content/uploads/2020/02/Kalimba.mp3"

# Download paths
mp3_path = "song.mp3"
wav_path = "song.wav"

# Download the MP3 file
r = requests.get(audio_url, allow_redirects=True)
r.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
with open(mp3_path, "wb") as f:
    f.write(r.content)

print(f"Downloaded MP3 to: {mp3_path}")

# Convert MP3 → WAV using FFmpeg (via pydub)
audio = AudioSegment.from_mp3(mp3_path)
audio.export(wav_path, format="wav")

print("Downloaded & converted to WAV:", wav_path)
```

```
Downloaded MP3 to: song.mp3
Downloaded & converted to WAV: song.wav
```

```
import librosa
from IPython.display import Audio, display

x, sr = librosa.load(wav_path, sr=22050, mono=True)

print("Audio Loaded:", len(x)/sr, "seconds")
display(Audio(x, rate=sr))
```

```
Audio Loaded: 348.05551020408166 seconds
0:00 / 5:48
```

◆ Gemini

```
import numpy as np
import matplotlib.pyplot as plt
import librosa.display
from scipy.signal import butter, lfilter

# Band-pass filter
def butter_bandpass_filter(data, lowcut, highcut, sr, order=6):
    nyq = sr / 2
    low = lowcut / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype="band")
    y = lfilter(b, a, data)
    return y

# Spectrogram function
def plot_spectrogram(sig, title):
    S = librosa.stft(sig)
    S_db = librosa.amplitude_to_db(np.abs(S) + 1e-10, ref=np.max)
    plt.figure(figsize=(10,3))
    # Explicitly set vmin and vmax to avoid issues with internal percentile calculation
    librosa.display.specshow(S_db, sr=sr, x_axis="time", y_axis="hz", vmin=-80, vmax=0)
    plt.title(title)
    plt.colorbar()
    plt.show()
```

```
from pydub import AudioSegment
import soundfile as sf
import numpy as np # Ensure numpy is imported for nan_to_num
```

```
# Band-pass ranges:
# Vocals ≈ 300–3400 Hz (telephone band)
# Music ≈ lower bass band 100–250 Hz

# 1. Extract VOCALS (300–3400 Hz)
vocals = butter_bandpass_filter(x, 300, 3400, sr)
plot_spectrogram(vocals, "Band-Pass Filter: Vocals (300–3400 Hz)")
display(Audio(vocals, rate=sr))

# Save as MP3
sf.write("vocals_filtered.wav", vocals, sr)
AudioSegment.from_wav("vocals_filtered.wav").export("vocals_filtered.mp3", format="mp3")
print("Saved: vocals_filtered.mp3")

# 2. Extract MUSIC (100–250 Hz → bass region)
music = butter_bandpass_filter(x, 100, 250, sr)

# Check for and handle non-finite values (NaN, Inf) before plotting
if not np.isfinite(music).all():
    print("Warning: Non-finite values detected in 'music' signal. Replacing with 0.")
    music = np.nan_to_num(music) # Replace NaN and Inf with 0

plot_spectrogram(music, "Band-Pass Filter: Music (100–250 Hz)")
display(Audio(music, rate=sr))

# Save as MP3
sf.write("music_filtered.wav", music, sr)
AudioSegment.from_wav("music_filtered.wav").export("music_filtered.mp3", format="mp3")
print("Saved: music_filtered.mp3")
```

